MM811 Programming Assignment

Oluwatimilehin Adenipekun - 1970644

October 2026

Abstract

This report presents the implementation and evaluation of Tasks 1-3 and 4.1 in the MM811 Programming Assignment. The report outlines the methods, implementation decisions, and experimental results of each task.

Contents

1 Task 1: Image Classification 2
- 1.1 Objective . . . 2
- 1.2 Dataset and Preprocessing . . . 2
- 1.3 Network Architecture . . . 2
  - 1.3.1 Convolutions and output dimensions . . . 3
  - 1.3.2 Model code . . . 3
- 1.4 Training Configuration . . . 3
  - 1.4.1 Training code . . . 4
- 1.5 Testing and Evaluation . . . 4
- 1.6 Experimental Results . . . 4
  - 1.6.1 Example digit predictions 5

2 Task 2: Binary Autoencoder 5
- 2.1 Objective . . . 5
- 2.2 Dataset and Preprocessing . . . 5
- 2.3 Autoencoder Architecture . . . 6
- 2.4 Binary Latent Representation . . 6
- 2.5 Decoder and Reconstruction . . . 7
- 2.6 Loss Function and PSNR . . . 7
- 2.7 Training Configuration . . . 7
- 2.8 Experimental Results . . . 8
- 2.9 Verification of Binary Representation . . . 8

3 Task 3: Autoregressive Models 9
- 3.1 Objective . . . 9
- 3.2 Dataset and Token Preparation . 9
- 3.3 Sequence-to-Token Model . . . 9
  - 3.3.1 Training . . . 10
  - 3.3.2 Experimental Results . . 10
- 3.4 Sequence-to-Sequence Model . . 11
  - 3.4.1 Training . . . 11
  - 3.4.2 Experimental Results . . 12
- 3.5 Sequence-to-Histogram Model . . 12
  - 3.5.1 Histogram Implementation 12
  - 3.5.2 Sequence Generation . . . 13
  - 3.5.3 Experimental Results . . 13
- 3.6 Overall Experimental Results . . 13

4 Task 4: Image Generation **14**
4.1 Question 4.1: Autoencoders . . . 14
4.1.1 Objective 14
4.1.2 Dataset and Preprocessing 14
4.1.3 Continuous Autoencoder . 15
4.1.4 Continuous Autoencoder Training 15
4.1.5 Continuous Autoencoder Results 15
4.1.6 Discrete Autoencoder . . 16
4.1.7 Uniform Quantization . . 16
4.1.8 Discrete Autoencoder Implementation 17
4.1.9 Discrete Autoencoder Training 17
4.1.10 Experimental Results . . 18

1 Task 1: Image Classification

1.1 Objective

The objective of this task was to implement an image classification model using the MNIST dataset to recognise handwritten digits from 0 to 9. A modified LeNet-5 convolutional neural network (CNN) was used to perform the classification, with the aim of achieving an accuracy above 90%.

1.2 Dataset and Preprocessing

MNIST contains 70,000 grayscale images of handwritten digits: 60,000 training images and 10,000 test images. Each image is 28×28 pixels. The images were loaded using `torchvision.datasets.MNIST`. The `ToTensor()` transform converts each image into a tensor and scales its pixel values from 0–255 to 0–1. This gives the model numerical inputs on a consistent scale.

I define a `load_mnist()` function. It uses a default batch size of five, but both the training and test loaders were actually created with a batch size of 64. Training data was shuffled, while test data was not. Shuffling changes the order of training examples between passes through the dataset. Each full input batch has shape $[64, 1, 28, 28]$: 64 images, one grayscale channel and 28-by-28 pixels.

Snippet 1: MNIST loading

```
def load_mnist(batch_size=5, train=True):
    transform = transforms.ToTensor()
    dataset = datasets.MNIST(
        root="./data",
        train=train,
        download=True,
        transform=transform
    )
    return DataLoader(dataset,
        batch_size=batch_size, shuffle=
        train)

train_loader = load_mnist(batch_size=64,
     train=True)
test_loader = load_mnist(batch_size=64,
    train=False)
```

1.3 Network Architecture

The model is based on LeNet-5. A CNN was chosen because pixels form meaningful patterns

in handwritten digits, including lines, edges, and curves. Convolutional filters examine small areas and reuse their learned weights across the image. Table 1 shows the network architecture. It also shows how the output dimensions change after each operation.

Table 1: Modified LeNet-5 network architecture (batch dimension omitted)

Layer	Configuration	Output shape
Input	Grayscale MNIST	$1 \times 28 \times 28$
Conv2d 1	1 to 6 channels; 5×5; stride 1; pad 2	$6 \times 28 \times 28$
Tanh	Activation	$6 \times 28 \times 28$
AvgPool2d 1	2×2; stride 2	$6 \times 14 \times 14$
Conv2d 2	6 to 16 channels; 5×5; stride 1	$16 \times 10 \times 10$
Tanh	Activation	$16 \times 10 \times 10$
AvgPool2d 2	2×2; stride 2	$16 \times 5 \times 5$
Flatten	$16 \times 5 \times 5$	400
Linear 1	$400 \to 120$	120
Tanh	Activation	120
Linear 2	$120 \to 84$	84
Tanh	Activation	84
Linear 3	$84 \to 10$	10

1.3.1 Convolutions and output dimensions

A convolutional filter is a small set of weights that moves across an image. The output size along one spatial dimension is

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1, \tag{1}$$

where I is input size, K kernel size, P padding and S stride. The first convolution uses $I = 28$, $K = 5$, $P = 2$, $S = 1$, so $O = (28-5+4)/1+1 = 28$. Padding preserves the image width and height while six filters learn six feature maps. After the first pooling operation, the spatial size is 14. The second convolution has no padding, so $O = (14 - 5)/1 + 1 = 10$, and its 16 filters produce 16 feature maps. Using more output channels lets later layers learn different combinations of earlier features.

1.3.2 Model code

Snippet 2: LeNet class from the Colab notebook

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()

        # Feature extraction
        self.feature = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size
                =5, stride=1, padding=2)
                ,
            nn.Tanh(),
            nn.AvgPool2d(kernel_size=2,
                stride=2),
            nn.Conv2d(6, 16, kernel_size
                =5, stride=1),
            nn.Tanh(),
            nn.AvgPool2d(kernel_size=2,
                stride=2)
        )

        # Classification
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120),
            nn.Tanh(),
            nn.Linear(120, 84),
            nn.Tanh(),
            nn.Linear(84, 10)
        )

    def forward(self, x):
        x = self.feature(x)
        x = self.classifier(x)
        return x
```

`nn.Module` provides the PyTorch model structure. `__init__()` defines the layers, while `forward()` sets the order in which data moves through them. `nn.Sequential` keeps the operations in a clear sequence. The model separates feature extraction from classification, making it easier to understand what each part does.

1.4 Training Configuration

Table 2: Experimental settings

Setting	Value
Dataset	MNIST
Training / test images	60,000 / 10,000
Framework	PyTorch
Model	Modified LeNet-5
Batch size	64
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Epochs	5
Device reported by notebook	CUDA GPU

An *epoch* is one complete pass through the training dataset. A batch size of 64 means the model processes 64 examples before updating its weights, except possibly in the last batch. Adam was used to adjust the weights based on gradients; it adapts its updates using estimates of recent gradients and their squared values. A learning rate of 0.001 controls the scale of these updates.

Cross-entropy loss measures how strongly the model favours the correct class. For a single example with correct class y and logits $z_1, \ldots, z_{10}$, it is

$$L = -\log\left(\frac{e^{z_y}}{\sum_{j=1}^{10} e^{z_j}}\right). \tag{2}$$

A smaller loss generally means the model assigns more confidence to the correct digit. This is appropriate because each image has one correct class among ten choices.

1.4.1 Training code

Snippet 3: Training setup and training loop from the notebook

```
device = torch.device("cuda" if torch.
    cuda.is_available() else "cpu")
model = LeNet().to(device)
learning_rate = 1e-3
epochs = 5
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.
    parameters(), lr=learning_rate)

def train_loop(dataloader, model,
    loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(
        dataloader):
        X, y = X.to(device), y.to(device
            )
        pred = model(X)
        loss = loss_fn(pred, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if batch % 100 == 0:
            loss_value = loss.item()
            current = min((batch + 1) *
                len(X), size)
            print(f"Loss: {loss_value:.4
                f} [{current}/{size}]")
```

The model and data are moved to the available device. `model.train()` sets training mode. The forward pass, `model(X)`, produces logits. The loss compares them with the labels. `zero_grad()` clears gradients left from the previous batch, `backward()` calculates new gradients through backpropagation, and `step()` updates the parameters. Progress is printed every 100 batches; these printed batch losses are not the average training loss for the entire epoch.

1.5 Testing and Evaluation

I use `model.eval()` to switch to evaluation mode and `torch.no_grad()` to avoid recording gradients during testing. It uses `argmax(1)` to select the class with the highest logit and compares it with the true label. Accuracy is the number of correct predictions divided by the total number of test images.

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total test images}} \times 100\%. \tag{3}$$

Snippet 4: Testing loop from the notebook

```
def test_loop(dataloader, model, loss_fn
    ):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0
    model.eval()
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(
                device)
            pred = model(X)
            test_loss += loss_fn(pred, y
                ).item()
            correct += (pred.argmax(1)
                == y).type(torch.float).
                sum().item()
    test_loss /= num_batches
    correct /= size
    print(f"Test Accuracy: {(100 *
        correct):.2f}%")
    print(f"Average Test Loss: {
        test_loss:.4f}")

for epoch in range(epochs):
    print(f"Epoch {epoch + 1}/{epochs}")
    print("
        -------------------------------"
        )
    train_loop(train_loader, model,
        loss_fn, optimizer)
    test_loop(test_loader, model,
        loss_fn)
print("Training Complete!")
```

The test loss in this implementation is the mean of the batch loss values, rather than an exact image-weighted mean. Because the last batch can be smaller, these two averages can differ slightly. The accuracy calculation does count individual correct predictions across all 10,000 test images.

1.6 Experimental Results

The model was tested after every epoch. Table 3 uses the complete training output.

Table 3: MNIST test results over five epochs

Epoch	Test accuracy (%)	Average test loss
1	95.94	0.1307
2	97.65	0.0791
3	98.22	0.0579
4	97.97	0.0677
5	**98.39**	**0.0513**

After the first epoch, the accuracy was already 95.94%, above the required 90%. It increased overall and reached 98.39% in the fifth epoch, 8.39 percentage points above the target. Test loss also fell overall from 0.1307 to 0.0513. There was a small drop in accuracy and increase in loss in epoch 4, but both measures improved in epoch 5. Model performance does not have to improve after every single epoch.

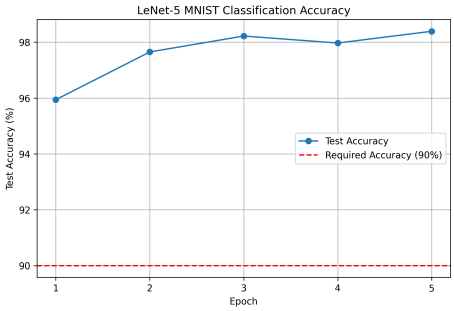


Figure 1: Test accuracy across five epochs, including the 90% target.

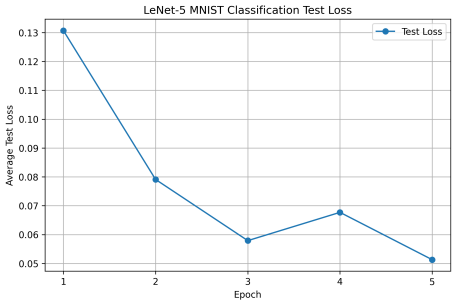


Figure 2: Average test loss across five epochs.

1.6.1 Example digit predictions

The function selects one test batch, runs the trained model, and displays ten images with their actual and predicted labels. Green titles indicate correct predictions; red titles indicate incorrect predictions. In the reported sample, nine were correct and one was incorrect.

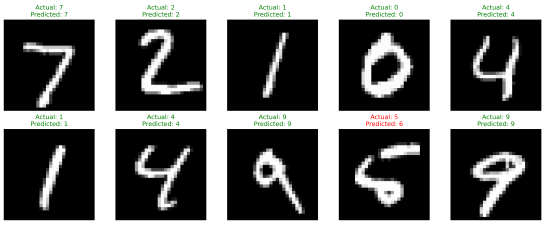


Figure 3: Example MNIST predictions compared with true labels.

2 Task 2: Binary Autoencoder

2.1 Objective

The objective of this task was to implement an autoencoder that compresses MNIST images into a binary latent representation of size $3 \times 4 \times 4$ and reconstructs the original images. The reconstruction quality was measured using Peak Signal-to-Noise Ratio (PSNR), with a target average PSNR greater than 25 dB.

2.2 Dataset and Preprocessing

The MNIST dataset was used again for this task. Each image is grayscale and has dimensions 28×28. The original pixel values range from 0 to 255, so they were divided by 255 to give values between 0 and 1.

A random subset of 256 images was selected from the 60,000 training images using a fixed random seed of 811. The official test set of 10,000 images was loaded separately for evaluation.

Snippet 5: Loading and normalising MNIST images

```
NUM_IMAGES = 256

mnist_train = datasets.MNIST(
    root='./data', train=True, download=
        True
)
mnist_test = datasets.MNIST(
    root='./data', train=False, download
        =True
)

def normalise_images(ds):
    return ds.data.float().div(255.0).
        unsqueeze(1)
```

```
12
13 full_train_images = normalise_images(
       mnist_train)
14
15 selected_indices = torch.randperm(
16     full_train_images.size(0),
17     generator=torch.Generator().
           manual_seed(SEED)
18 )
19
20 train_images = full_train_images[
21     selected_indices[:NUM_IMAGES]
22 ].to(device)
23
24 test_images = normalise_images(
       mnist_test).to(device)
```

The `unsqueeze(1)` operation adds the channel dimension required by PyTorch convolutional layers. Each image therefore has shape $1 \times 28 \times 28$.

The fixed seed makes the selection of training images reproducible. This allows the same subset to be used when repeating the experiment.

2.3 Autoencoder Architecture

An autoencoder is a neural network that learns to reconstruct its input. It consists of an encoder, a compressed latent representation, and a decoder. The implemented model uses convolutional layers in the encoder and transposed convolutional layers in the decoder.

Table 4: Binary autoencoder architecture

Stage	Operation	Output shape
Input	MNIST image	$1 \times 28 \times 28$
Encoder	Conv blocks, 64 channels	$64 \times 28 \times 28$
Encoder	Conv blocks, 128 channels	$128 \times 14 \times 14$
Encoder	Conv blocks, 256 channels	$256 \times 7 \times 7$
Projection	Fully connected layers	48
Reshape	Latent representation	$3 \times 4 \times 4$
Binarization	BinarySTE	$3 \times 4 \times 4$
Decoder	Fully connected projection	$256 \times 7 \times 7$
Decoder	Transposed convolution	$128 \times 14 \times 14$
Decoder	Transposed convolution	$64 \times 28 \times 28$
Output	Conv2d and Sigmoid	$1 \times 28 \times 28$

The convolutional blocks use 3×3 kernels, batch normalization, and ReLU activation. Padding of 1 preserves spatial dimensions when the stride is 1. A stride of 2 is used to reduce the spatial dimensions from 28 to 14 and then from 14 to 7.

The output size of a convolution is calculated using:

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1 \tag{4}$$

where I is the input size, K is the kernel size, P is padding, and S is stride.

For the first downsampling convolution:

$$O = \left\lfloor \frac{28 - 3 + 2(1)}{2} \right\rfloor + 1 = 14 \tag{5}$$

The encoder eventually produces 256 feature maps of size 7×7. These are flattened and passed through fully connected layers to produce 48 values.

$$3 \times 4 \times 4 = 48, \tag{6}$$

Snippet 6: Encoder latent projection

```
1  self.to_latent = nn.Sequential(
2      nn.Flatten(),
3      nn.Linear(256 * 7 * 7, 512, bias=
           False),
4      nn.BatchNorm1d(512),
5      nn.ReLU(inplace=True),
6      nn.Linear(512, 48),
7  )
8
9  self.latent_norm = nn.BatchNorm1d(48)
10
11 def forward(self, x):
12     z = self.latent_norm(
13         self.to_latent(self.features(x))
14     )
15     return z.view(-1, 3, 4, 4)
```

Batch normalization is used before binarization to help keep the latent values centred around zero, which is the threshold used to assign binary values.

2.4 Binary Latent Representation

The latent representation must contain binary values rather than continuous values.

This implementation uses signed binary values, -1 and $+1$. A sign-based operation converts values greater than or equal to zero into $+1$, and negative values into -1.

$$b(x) = \begin{cases} +1, & x \geq 0 \\ -1, & x < 0 \end{cases} \tag{7}$$

A Straight-Through Estimator (STE) was therefore implemented. During the forward pass, it produces binary values. During the backward pass, it approximates the gradient so that the encoder can continue learning.

Snippet 7: Binary Straight-Through Estimator

```
1  class BinarizeFn(torch.autograd.Function
       ):
2      @staticmethod
```

```
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return torch.where(
            x >= 0,
            torch.ones_like(x),
            -torch.ones_like(x)
        )

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        return grad_out * (
            x.abs() <= 1
        ).to(grad_out.dtype)

class BinarySTE(nn.Module):
    def forward(self, x):
        return BinarizeFn.apply(x)
```

The backward function passes gradients through values in the interval $[-1, 1]$ and blocks gradients outside this interval. This is an approximation rather than the true derivative of the sign operation.

We verified that the binary latent tensor had dimensions $(8, 3, 4, 4)$ for a batch of eight images, and that its only values were -1 and $+1$.

2.5 Decoder and Reconstruction

The decoder converts the binary representation back into an image.

The 48 latent values are first passed through fully connected layers and reshaped into 256 feature maps of size 7×7.

Transposed convolutions then increase the spatial dimensions from 7×7 to 14×14, and finally to 28×28.

Snippet 8: Decoder upsampling layers

```
self.up = nn.Sequential(
    nn.ConvTranspose2d(
        256, 128, 4, stride=2,
        padding=1, bias=False
    ),
    nn.BatchNorm2d(128),
    nn.ReLU(inplace=True),
    *conv_block(128, 128),

    nn.ConvTranspose2d(
        128, 64, 4, stride=2,
        padding=1, bias=False
    ),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    *conv_block(64, 64),

    nn.Conv2d(64, 1, 3, padding=1),
    nn.Sigmoid()
)
```

The final convolution produces one channel because MNIST images are grayscale. The Sigmoid activation limits the reconstructed pixel values to the range $[0, 1]$, matching the normalized input images.

2.6 Loss Function and PSNR

Reconstruction quality was measured using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR).

MSE measures the average squared difference between the original image and its reconstruction:

$$MSE = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} (x_{ij} - \hat{x}_{ij})^2 \qquad (8)$$

where x is the original image and $\hat{x}$ is the reconstructed image.

For images normalized to $[0, 1]$, PSNR is calculated as:

$$PSNR = 10 \log_{10} \left(\frac{1}{MSE} \right) \qquad (9)$$

Snippet 9: Reconstruction loss and PSNR calculation

```
def calculate_image_mse(recon, x):
    return ((recon - x) ** 2).mean(
        dim=(1, 2, 3)
    )

def psnr_loss(recon, x, eps=1e-5):
    return torch.log(
        calculate_image_mse(recon, x) +
            eps
    ).mean()
```

Minimizing this loss encourages lower reconstruction error. The logarithm also makes the training objective closely related to maximizing PSNR.

The evaluation function calculates PSNR separately for each image before taking the average. This is why average PSNR should not be calculated directly from the reported average MSE.

2.7 Training Configuration

The model was trained using Adam with an initial configured learning rate of 0.002. A OneCycleLR scheduler was used to adjust the learning rate during training.

The model was trained for 1,500 epochs because only 256 training images were selected.

Table 5: Task 2 experimental configuration

Setting	Value
Dataset	MNIST
Training images	256
Held-out test images	10,000
Batch size	64
Epochs	1,500
Optimizer	Adam
Maximum learning rate	0.002
Learning rate scheduler	OneCycleLR
Training objective	Mean log-MSE
Binary latent dimensions	$3 \times 4 \times 4$
Latent values	$-1, +1$
Hardware	CUDA GPU

Each epoch processed four batches of 64 images.

The OneCycleLR scheduler first increases and then decreases the learning rate. This allows the optimizer to make larger updates during part of training and smaller updates later.

Snippet 10: Optimizer and learning rate scheduler

```
optimizer = optim.Adam(
    model.parameters(), lr=LEARNING_RATE
)

steps_per_epoch = (
    train_images.size(0) // BATCH_SIZE
)

scheduler = optim.lr_scheduler.
    OneCycleLR(
    optimizer,
    max_lr=LEARNING_RATE,
    epochs=NUM_EPOCHS,
    steps_per_epoch=steps_per_epoch,
    pct_start=0.1,
    anneal_strategy='cos',
    final_div_factor=1e3
)
```

During each batch, the model reconstructed the images, calculated the loss, performed backpropagation, and updated the weights.

The model checkpoint with the highest PSNR on the training subset was saved.

2.8 Experimental Results

The best checkpoint was recorded at epoch 1483. The average PSNR on the 256-image training subset was 66.11 dB, exceeding the 25 dB threshold for those images.

Table 6: Binary autoencoder evaluation results

Measurement	Result
Best checkpoint epoch	1483
Training-subset PSNR	66.11 dB
Training-subset MSE	Approximately 0.000001
Worst training-subset PSNR	38.16 dB
Training-subset median PSNR	66.22 dB
Training-subset target exceeded	Yes

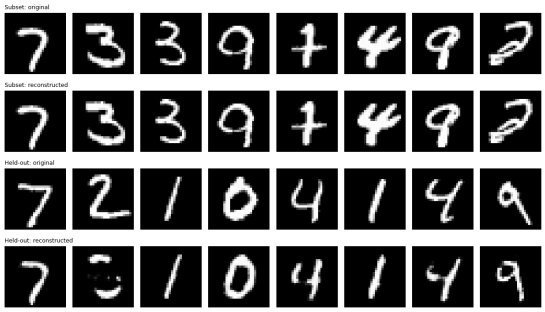


Figure 4: Original and reconstructed MNIST images from the training subset and held-out test set.

The training curves show that the subset PSNR increased while the subset MSE decreased over 1,500 epochs.

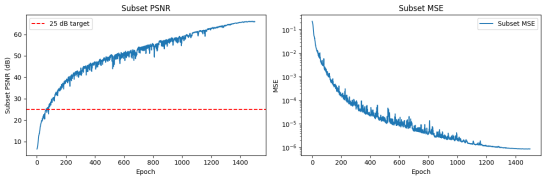


Figure 5: Training-subset PSNR and MSE over 1,500 epochs.

The original and reconstructed images were also displayed for both the training subset and the held-out test set.

2.9 Verification of Binary Representation

The binary latent representation was checked after training to confirm that it satisfied the required dimensions and binary-value constraint.

Snippet 11: Verification of binary latent codes

```
codes = []

with torch.no_grad():
```

```
    for xb in get_batches(train_images,
        500):
        codes.append(
            model.encode(xb)
            .flatten(start_dim=1).cpu()
        )

codes = torch.cat(codes)

print(torch.unique(codes).tolist())
print(
    ((codes != -1) & (codes != 1))
    .sum().item()
)
```

Table 7: Binary latent representation verification

Measurement	Result
Latent dimensions per image	$3 \times 4 \times 4$
Values per image	48
Allowed values	$-1, +1$
Non-binary entries	0
Unique codes	256 of 256
Active bits	48 of 48
Model parameters	14.91 million

3 Task 3: Autoregressive Models

3.1 Objective

The objective of this task was to implement three autoregressive models using binary representations of MNIST images.

3.2 Dataset and Token Preparation

The trained binary autoencoder from Task 2 was used to convert MNIST images into binary latent representations.

For the first two models, 100 MNIST training images were selected. The images were normalized to the range $[0, 1]$ and passed through the encoder.

Snippet 12: Loading the trained autoencoder

```
model = BinaryAutoencoder().to(device)

checkpoint = torch.load(
    checkpoint_path,
    map_location=device,
    weights_only=True
)

model.load_state_dict(
    checkpoint['model_state_dict']
)
model.eval()
```

The encoder produces a binary representation of size $3 \times 4 \times 4$ for each image. Each of the three channels contains either -1 or $+1$.

These signed binary values were first converted into ordinary binary values using:

$$b = \frac{z + 1}{2} \tag{10}$$

This converts -1 into 0 and $+1$ into 1.

The three binary values at each spatial position were then combined into a decimal token:

$$t = 4b_1 + 2b_2 + b_3 \tag{11}$$

For example, the binary combination $(1, 0, 1)$ gives:

$$t = 4(1) + 2(0) + 1 = 5 \tag{12}$$

Since three bits can represent $2^3 = 8$ combinations, the resulting tokens range from 0 to 7. Each latent representation contains a 4×4 grid of spatial positions. Flattening this grid produces a sequence of 16 tokens per image.

Snippet 13: Converting binary latents into token sequences

```
with torch.no_grad():
    binary_latents = model.encode(images
        )

binary_bits = (
    (binary_latents + 1) / 2
).long()

tokens = (
    binary_bits[:, 0] * 4
    + binary_bits[:, 1] * 2
    + binary_bits[:, 2]
)

token_sequences = tokens.reshape(
    NUM_IMAGES, 16
)
```

The resulting tensor had dimensions $[100, 16]$, representing 100 images with 16 tokens each.

3.3 Sequence-to-Token Model

The first autoregressive model was implemented using a Gated Recurrent Unit (GRU).

Table 8: Token preparation and dimensions

Stage	Operation	Output shape
Input	MNIST images	$100 \times 1 \times 28 \times 28$
Encoder	Binary latent	$100 \times 3 \times 4 \times 4$
Conversion	Signed to binary	$100 \times 3 \times 4 \times 4$
Token mapping	Three bits to one token	$100 \times 4 \times 4$
Flatten	Token sequences	100×16

A GRU is a recurrent neural network that processes a sequence while maintaining information from previous tokens. It was used because the prediction of each token depends on the tokens that came before it.

The model consists of an embedding layer, a GRU layer, and a fully connected output layer.

Snippet 14: Sequence-to-Token GRU architecture

```
class SequenceToToken(nn.Module):
    def __init__(self):
        super().__init__()

        self.embedding = nn.Embedding(9,
            64)
        self.gru = nn.GRU(
            64, 256, batch_first=True
        )
        self.fc = nn.Linear(256, 8)

    def forward(self, x):
        x = self.embedding(x)
        output, _ = self.gru(x)
        return self.fc(output)
```

Table 9: Sequence-to-Token network architecture

Layer	Configuration	Output shape
Input	Token sequence	$N \times 16$
Embedding	9 tokens, 64 features	$N \times 16 \times 64$
GRU	Hidden size 256	$N \times 16 \times 256$
Linear	$256 \to 8$	$N \times 16 \times 8$

3.3.1 Training

A start token was added before each training sequence. The last token was removed from the input so that the input and target sequences both contained 16 positions.

The model was therefore trained to predict the next token at every position.

Snippet 15: Preparing autoregressive training inputs

```
START_TOKEN = 8
NUM_EPOCHS = 500

training_sequences = token_sequences.
    long()

start_tokens = torch.full(
    (training_sequences.size(0), 1),
    START_TOKEN,
    dtype=torch.long,
    device=device
)

inputs = torch.cat(
    [start_tokens, training_sequences[:,
        :-1]],
    dim=1
)

targets = training_sequences
```

The model was trained for 500 epochs using the Adam optimizer with a learning rate of 0.003.

Cross-Entropy Loss was used because the model predicts one of eight possible token classes at each position.

Snippet 16: Sequence-to-Token training

```
optimizer = optim.Adam(
    sequence_model.parameters(),
    lr=0.003
)

criterion = nn.CrossEntropyLoss()

for epoch in range(1, NUM_EPOCHS + 1):
    sequence_model.train()

    predictions = sequence_model(inputs)

    loss = criterion(
        predictions.reshape(-1, 8),
        targets.reshape(-1)
    )

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

The prediction tensor was reshaped because CrossEntropyLoss requires the class dimension to be separated from the target labels. The 100 sequences each contain 16 positions, giving 1,600 token predictions.

3.3.2 Experimental Results

The final recorded training loss was 0.2881, with a token prediction accuracy of 87.31%. The trained model was then used to generate complete sequences by repeatedly predicting the next token. Two generation methods were tested. Greedy generation selected the

token with the highest predicted score, while probability sampling selected tokens according to their predicted probabilities. Greedy generation produced 100 sequences, all of which matched sequences in the training set. For probability sampling, the model generated 1,000 sequences. Of these, 997 matched training sequences.

Memorization was calculated using:

$$M = \frac{N_{\text{match}}}{N_{\text{generated}}} \times 100 \tag{13}$$

For the sampled sequences:

$$M = \frac{997}{1000} \times 100 = 99.70\% \tag{14}$$

Table 10: Sequence-to-Token experimental results

Measurement	Result
Training sequences	100
Training epochs	500
Learning rate	0.003
Final training loss	0.2881
Final token accuracy	87.31%
Greedy memorization	100.00%
Sampled sequences	1,000
Unique sampled sequences	103
Matching sampled sequences	997
Sampled memorization	99.70%
Required memorization	$> 80\%$

The model exceeded the required memorization rate. The sampled result was used as the main measurement because it evaluates sequences generated from the predicted probability distribution rather than always choosing the most likely token.

3.4 Sequence-to-Sequence Model

The second autoregressive model was implemented using a Transformer encoder with causal attention. A causal attention mask was applied to prevent the model from accessing future tokens when predicting the next token.

Snippet 17: Sequence-to-Sequence Transformer architecture

```
class SequenceToSequence(nn.Module):
    def __init__(self):
        super().__init__()

        self.embedding = nn.Embedding(9,
             64)
        self.position_embedding = nn.
            Embedding(16, 64)

        layer = nn.
            TransformerEncoderLayer(
            d_model=64,
            nhead=4,
            dim_feedforward=256,
            dropout=0.0,
            batch_first=True
        )

        self.transformer = nn.
            TransformerEncoder(
            layer, num_layers=3
        )
        self.fc = nn.Linear(64, 8)

    def forward(self, x):
        positions = torch.arange(
            x.size(1), device=x.device
        )

        x = (
            self.embedding(x)
            + self.position_embedding(
                positions)
        )

        mask = torch.triu(
            torch.ones(
                x.size(1), x.size(1),
                device=x.device,
                dtype=torch.bool
            ),
            diagonal=1
        )

        x = self.transformer(x, mask=
            mask)
        return self.fc(x)
```

Table 11: Sequence-to-Sequence network architecture

Layer	Configuration	Output shape
Input	Token sequence	$N \times 16$
Embedding	64 features	$N \times 16 \times 64$
Position embedding	16 positions	$N \times 16 \times 64$
Transformer	3 layers, 4 heads	$N \times 16 \times 64$
Linear	$64 \to 8$	$N \times 16 \times 8$

3.4.1 Training

The Transformer was trained using the same input and target sequences as the GRU.

Adam was used with a learning rate of 0.001, and Cross-Entropy Loss was used to measure next-token prediction error.

Snippet 18: Sequence-to-Sequence training

```
NUM_EPOCHS = 500

optimizer = optim.Adam(
    sequence_to_sequence.parameters(),
    lr=0.001
)

criterion = nn.CrossEntropyLoss()

for epoch in range(1, NUM_EPOCHS + 1):
    sequence_to_sequence.train()

    predictions = sequence_to_sequence(
        inputs)

    loss = criterion(
        predictions.reshape(-1, 8),
        targets.reshape(-1)
    )

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

The model was trained for 500 epochs. The final recorded loss was 0.2890, and the token prediction accuracy was 87.31%.

3.4.2 Experimental Results

The Transformer generated 1,000 sequences by sampling from its predicted token probabilities.

Of these, 983 matched sequences from the training set.

$$M = \frac{983}{1000} \times 100 = 98.30\% \tag{15}$$

Table 12: Sequence-to-Sequence experimental results

Measurement	Result
Training sequences	100
Training epochs	500
Learning rate	0.001
Final training loss	0.2890
Final token accuracy	87.31%
Generated sequences	1,000
Unique generated sequences	117
Matching training sequences	983
Memorization rate	98.30%
Required memorization	$> 80\%$

The model exceeded the required memorization rate. The results show that the Transformer learned to generate sequences that closely matched the training data.

3.5 Sequence-to-Histogram Model

The third model used conditional histograms instead of a neural network.

A histogram records how often each token occurs after a particular sequence of previous tokens. These counts are converted into probabilities and used to generate the next token.

For this model, 600 MNIST training images were encoded using the binary autoencoder. The resulting representations were converted into sequences of 16 tokens using the same method as the previous models.

Snippet 19: Preparing histogram training sequences

```
NUM_HISTOGRAM_IMAGES = 600

histogram_images = (
    mnist.data[:NUM_HISTOGRAM_IMAGES]
    .float()
    .div(255.0)
    .unsqueeze(1)
    .to(device)
)

with torch.no_grad():
    histogram_latents = model.encode(
        histogram_images
    )

histogram_bits = (
    (histogram_latents + 1) / 2
).long()

histogram_tokens = (
    histogram_bits[:, 0] * 4
    + histogram_bits[:, 1] * 2
    + histogram_bits[:, 2]
)

histogram_sequences = histogram_tokens.
    reshape(
    NUM_HISTOGRAM_IMAGES, 16
)
```

3.5.1 Histogram Implementation

A dictionary was used to store the token counts for each observed prefix.

For every training sequence, the model recorded the next token associated with each prefix.

Snippet 20: Conditional histogram model

```
class SequenceToHistogram:
    def __init__(self):
        self.histograms = defaultdict(
            lambda: np.zeros(8, dtype=np
                .int64)
```

```
        )

    def fit(self, sequences):
        for sequence in sequences:
            sequence = sequence.tolist()

            for i in range(len(sequence)
                ):
                prefix = tuple(sequence
                    [:i])
                next_token = sequence[i]

                self.histograms[prefix][
                    next_token] += 1

    def predict(self, prefix):
        counts = self.histograms[tuple(
            prefix)]

        if counts.sum() == 0:
            raise ValueError(
                'Prefix was not observed
                     during training.'
            )

        return counts / counts.sum()
```

The `fit()` function builds the histograms by counting the next tokens for each observed prefix.

The `predict()` function converts these counts into probabilities:

$$P(t \mid p) = \frac{C(p,t)}{\sum_{j=0}^{7} C(p,j)} \tag{16}$$

where $C(p,t)$ is the number of times token t follows prefix p.

For example, if a prefix is followed by token 2 three times and token 5 once, the probabilities are:

$$P(2 \mid p) = \frac{3}{4} = 0.75 \tag{17}$$

$$P(5 \mid p) = \frac{1}{4} = 0.25 \tag{18}$$

Unlike the GRU and Transformer, this model does not use backpropagation or an optimizer. It learns directly from the observed token frequencies.

3.5.2 Sequence Generation

New sequences were generated by repeatedly sampling from the probability distribution associated with the current prefix.

Snippet 21: Generating sequences using histograms

```
def generate_histogram_sequences(
    model, num_sequences=1000
):
    generated = []

    for _ in range(num_sequences):
        sequence = []

        for _ in range(16):
            probabilities = model.
                predict(
                sequence
            )

            next_token = np.random.
                choice(
                8, p=probabilities
            )

            sequence.append(int(
                next_token))

        generated.append(sequence)

    return torch.tensor(
        generated, dtype=torch.long
    )
```

The generation begins with an empty prefix. After each token is selected, it is added to the sequence and used as part of the prefix for the next prediction.

Because the histograms were constructed from complete training prefixes, the model only samples tokens that were observed after the current prefix. This keeps the generated sequences on paths recorded in the training data.

3.5.3 Experimental Results

The histogram model was trained using 600 token sequences and produced 7,323 prefix histograms.

It generated 1,000 sequences, all of which matched complete sequences in the training set.

$$M = \frac{1000}{1000} \times 100 = 100.00\% \tag{19}$$

The model exceeded the required memorization threshold. Its 100% result follows from sampling only observed continuations of training prefixes.

3.6 Overall Experimental Results

Table 14 summarizes the results of the three autoregressive models.

All three models achieved their required memorization rates.

Table 13: Sequence-to-Histogram experimental results

Measurement	Result
Training images	600
Sequence length	16
Possible token values	0–7
Learned prefix histograms	7,323
Generated sequences	1,000
Unique generated sequences	479
Matching training sequences	1,000
Memorization rate	100.00%
Required memorization	$> 99\%$

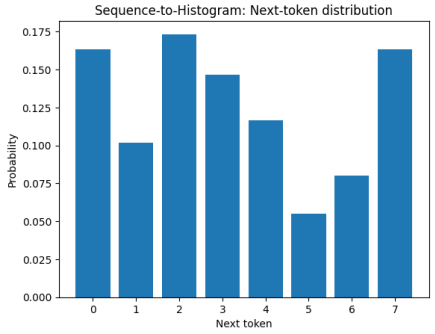


Figure 6: Sequence-to-Histogram next-token probability distribution for an empty prefix.

Table 14: Task 3 memorization results

Model	Matches	Memorization	Target
Sequence-to-Token	997/1000	99.70%	$> 80\%$
Sequence-to-Sequence	983/1000	98.30%	$> 80\%$
Sequence-to-Histogram	1000/1000	100.00%	$> 99\%$

The GRU and Transformer learned probability distributions through neural network training, while the histogram model used counts of observed token sequences. The memorization results measure how often complete generated sequences matched the training set, rather than performance on unseen images.

4 Task 4: Image Generation

4.1 Question 4.1: Autoencoders

4.1.1 Objective

The objective of Question 4.1 was to implement continuous and discrete autoencoders for reconstructing CIFAR-10 images. Both models were trained to compress an input image into a smaller latent representation and reconstruct the original image. Reconstruction quality was measured using Peak Signal-to-Noise Ratio (PSNR).

4.1.2 Dataset and Preprocessing

The CIFAR-10 dataset contains 60,000 colour images, divided into 50,000 training images and 10,000 test images. Each image has three RGB channels and a resolution of 32×32 pixels.

The dataset was loaded using `torchvision.datasets.CIFAR10`. The `ToTensor()` transformation converted pixel values from the range 0–255 to floating-point values between 0 and 1.

Snippet 22: Loading the CIFAR-10 dataset

```
transform = transforms.ToTensor()

train_dataset = datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

BATCH_SIZE = 128

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
```

```
    shuffle=True,
    num_workers=2
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2
)
```

A batch size of 128 was used for both training and testing. The training loader shuffled the images to vary their order during training, while the test loader preserved the evaluation order.

Each full batch had dimensions $128 \times 3 \times 32 \times 32$.

4.1.3 Continuous Autoencoder

The continuous autoencoder was implemented using convolutional layers for encoding and transposed convolutional layers for decoding.

The encoder reduces the spatial dimensions of the input image while increasing the number of feature channels. The decoder reverses this process to reconstruct an image with the original dimensions.

The model uses a continuous latent representation, meaning the encoded values are floating-point numbers rather than values restricted to a fixed set.

Table 15: Continuous autoencoder architecture

Stage	Operation	Output shape
Input	RGB image	$3 \times 32 \times 32$
Encoder	Convolutional layers	Reduced features
Latent	Continuous representation	Encoded features
Decoder	Transposed convolutions	Reconstructed features
Output	RGB reconstruction	$3 \times 32 \times 32$

Convolution was used because it can extract spatial features such as edges, colours and textures from images. Reducing the spatial dimensions allows the encoder to represent the image using a smaller set of features.

Transposed convolutions were used in the decoder to increase the spatial dimensions and recover the original image size.

4.1.4 Continuous Autoencoder Training

The continuous autoencoder was trained for 20 epochs using the Adam optimizer with a learning rate of 0.001.

Mean Squared Error (MSE) was used as the reconstruction loss. It measures the average squared difference between the original and reconstructed pixel values.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2 \tag{20}$$

where x_i represents an original pixel value and $\hat{x}_i$ represents the corresponding reconstructed value.

Snippet 23: Continuous autoencoder training

```
criterion = nn.MSELoss()

optimizer = optim.Adam(
    continuous_model.parameters(),
    lr=0.001
)

for epoch in range(1, NUM_EPOCHS + 1):
    continuous_model.train()
    total_loss = 0.0

    for images_batch, _ in train_loader:
        images_batch = images_batch.to(
            device)

        reconstructed, latent = (
            continuous_model(
                images_batch)
        )

        loss = criterion(
            reconstructed, images_batch
        )

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += (
            loss.item() * images_batch.
                size(0)
        )

    avg_loss = total_loss / len(
        train_dataset)
```

The training loop passes each image batch through the autoencoder, calculates the reconstruction error and updates the model parameters using backpropagation.

The model was evaluated on the CIFAR-10 test set at selected epochs. The checkpoint with the highest recorded test PSNR was saved.

4.1.5 Continuous Autoencoder Results

PSNR was calculated using the mean squared reconstruction error. Since the images were normalized to the range $[0, 1]$, the maximum possible pixel intensity was 1.

$$\mathrm{PSNR} = 10 \log_{10} \left(\frac{1}{\mathrm{MSE}} \right) \tag{21}$$

A higher PSNR indicates a lower reconstruction error.

Table 16: Continuous autoencoder training results

Epoch	Training MSE	Test PSNR (dB)
1	0.011547	23.68
5	0.002403	26.34
10	0.002080	24.86
15	0.001942	27.23
20	0.001850	27.31

The continuous autoencoder achieved its highest recorded test PSNR of 27.31 dB at epoch 20. The training loss decreased from 0.011547 to 0.001850.

The test PSNR was not strictly increasing. For example, it decreased at epoch 10 before improving at epochs 15 and 20. This shows that a lower training loss does not always produce a higher test PSNR at every evaluation.

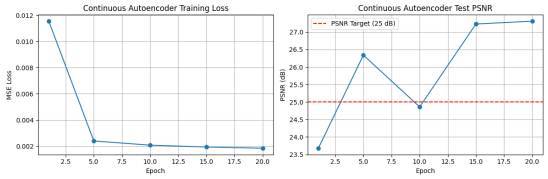


Figure 7: Continuous autoencoder training loss and test PSNR.



Figure 8: Original CIFAR-10 images and continuous autoencoder reconstructions.

4.1.6 Discrete Autoencoder

The second autoencoder was designed to use a discrete latent representation.

Unlike a continuous autoencoder, which can produce arbitrary floating-point latent values, a discrete autoencoder restricts its latent values to a finite set.

The implemented model used 16 quantization levels. Its encoder produced a latent tensor with 10 channels and spatial dimensions 8×8.

For a batch of N images, the latent representation had dimensions:

$$N \times 10 \times 8 \times 8 \tag{22}$$

Each image therefore contained:

$$10 \times 8 \times 8 = 640 \tag{23}$$

latent values, with each value restricted to one of 16 possible levels.

Table 17: Discrete autoencoder architecture

Stage	Configuration	Output shape
Input	CIFAR-10 RGB	$3 \times 32 \times 32$
Conv2d	3 to 64 channels	$64 \times 32 \times 32$
Residual block	64 channels	$64 \times 32 \times 32$
Conv2d	64 to 128, stride 2	$128 \times 16 \times 16$
Residual block	128 channels	$128 \times 16 \times 16$
Conv2d	128 to 256, stride 2	$256 \times 8 \times 8$
Residual block	256 channels	$256 \times 8 \times 8$
Conv2d	256 to 10	$10 \times 8 \times 8$
Quantizer	16 levels	$10 \times 8 \times 8$
Conv2d	10 to 256	$256 \times 8 \times 8$
Residual block	256 channels	$256 \times 8 \times 8$
ConvTranspose2d	256 to 128	$128 \times 16 \times 16$
Residual block	128 channels	$128 \times 16 \times 16$
ConvTranspose2d	128 to 64	$64 \times 32 \times 32$
Residual block	64 channels	$64 \times 32 \times 32$
Conv2d	64 to 3	$3 \times 32 \times 32$
Sigmoid	Output activation	$3 \times 32 \times 32$

Residual blocks were used to allow information to pass through convolutional layers using skip connections. The model also used Group Normalization and SiLU activation functions.

The decoder used transposed convolutions to increase the spatial dimensions from 8×8 back to 32×32.

A final Sigmoid activation restricted reconstructed pixel values to the range $[0, 1]$, matching the normalized input images.

4.1.7 Uniform Quantization

The discrete representation was produced using a uniform quantizer with 16 levels.

The encoder output was first passed through the Tanh function to restrict the values to the range $[-1, 1]$.

The values were then rounded to one of 16 equally spaced levels.

Snippet 24: Uniform quantizer with straight-through estimator

```python
class UniformQuantizer(nn.Module):
    def __init__(self, levels=16):
        super().__init__()
        self.levels = levels

    def forward(self, x):
        x = torch.tanh(x)

        quantized = torch.round(
            (x + 1) * (self.levels - 1)
                / 2
        )

        quantized = (
            quantized * 2 / (self.levels
                 - 1) - 1
        )

        return x + (quantized - x).
            detach()
```

The rounding operation is not differentiable in the usual way because small changes in the input often do not change the rounded output.

This creates a problem for backpropagation, which relies on gradients to update the model parameters.

A Straight-Through Estimator (STE) was therefore used.

The expression:

$$y = x + \text{detach}(q(x) - x) \tag{24}$$

returns the quantized value during the forward pass while allowing gradients to pass through the original continuous value during backpropagation.

This allows the autoencoder to learn while maintaining a discrete latent representation.

4.1.8 Discrete Autoencoder Implementation

The discrete autoencoder passes the input through the encoder, applies quantization and reconstructs the image using the decoder.

Snippet 25: Discrete autoencoder forward operation

```python
def forward(self, x):
    latent = self.encoder(x)

    discrete_latent = self.quantizer(
        latent)

    reconstructed = self.decoder(
        discrete_latent
    )

    return reconstructed,
        discrete_latent
```

The implementation was checked using four randomly generated RGB images.

The output dimensions were:

Table 18: Discrete autoencoder dimension verification

Measurement	Output
Input	$4 \times 3 \times 32 \times 32$
Latent	$4 \times 10 \times 8 \times 8$
Reconstruction	$4 \times 3 \times 32 \times 32$
Unique values observed	14
Configured quantization levels	16

The test confirmed that the model reduced the input to the intended latent dimensions and reconstructed images with the original dimensions.

Although the quantizer supported 16 levels, only 14 distinct values appeared in this particular random test. The number of observed levels can be smaller than the number of available levels.

4.1.9 Discrete Autoencoder Training

The discrete autoencoder was trained for 30 epochs using the AdamW optimizer.

The learning rate was set to 0.0005, with weight decay of 0.00001. A CosineAnnealingLR scheduler was used to reduce the learning rate over training.

Mean Squared Error was again used as the reconstruction loss.

Snippet 26: Discrete autoencoder training configuration

```python
NUM_EPOCHS = 30

criterion = nn.MSELoss()

optimizer = optim.AdamW(
    discrete_model.parameters(),
    lr=0.0005,
    weight_decay=0.00001
)

scheduler = (
    optim.lr_scheduler.CosineAnnealingLR
        (
        optimizer,
        T_max=NUM_EPOCHS
    )
)
```

AdamW was used to update the network weights while applying weight decay. The learning rate scheduler gradually reduced the learn-

ing rate, allowing smaller parameter updates later in training.

The training loop reconstructed each image batch, calculated MSE, performed backpropagation and updated the model parameters.

Snippet 27: Discrete autoencoder parameter updates

```
for images_batch, _ in train_loader:
    images_batch = images_batch.to(
        device)

    reconstructed, latent = (
        discrete_model(images_batch)
    )

    loss = criterion(
        reconstructed, images_batch
    )

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += (
        loss.item() * images_batch.size
            (0)
    )
```

After each epoch, the model was evaluated on the test dataset. The checkpoint with the highest test PSNR was saved.

4.1.10 Experimental Results

The discrete autoencoder was trained for 30 epochs and achieved a best recorded test PSNR of 31.59 dB.

Table 19: Question 4.1 training configuration

Setting	Continuous	Discrete
Dataset	CIFAR-10	CIFAR-10
Training images	50,000	50,000
Test images	10,000	10,000
Batch size	128	128
Epochs	20	30
Optimizer	Adam	AdamW
Learning rate	0.001	0.0005
Loss	MSE	MSE
Latent type	Continuous	Discrete
Quantization	None	16 levels
Best test PSNR	27.31 dB	31.59 dB

The difference between the best recorded PSNR values was:

$$31.59 - 27.31 = 4.28 \text{ dB} \tag{25}$$

The discrete autoencoder achieved a higher recorded PSNR than the continuous autoencoder in these experiments. However, the models used different architectures, optimizers and training durations, so this result does not establish that discrete representations are always better than continuous representations.

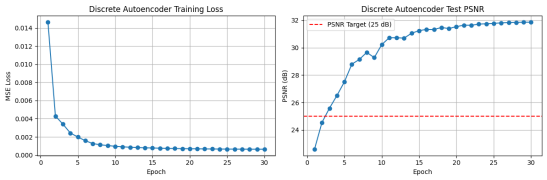


Figure 9: Discrete autoencoder training loss and test PSNR.



Figure 10: Original CIFAR-10 images and discrete autoencoder reconstructions.

References

[1] PyTorch, *Datasets and DataLoaders*. https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html

[2] PyTorch, *Build the Neural Network*. https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html

[3] PyTorch, *Optimizing Model Parameters*. https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[4] PyTorch, *Save and Load the Model*. https://docs.pytorch.org/tutorials/beginner/basics/saveloadrun_tutorial.html

[5] GeeksforGeeks, *MNIST Dataset*. https://www.geeksforgeeks.org/machine-learning/mnist-dataset/

[6] Deepesh Deepak, *LeNet-5 Implementation on MNIST in PyTorch*. https://medium.com/@deepeshdeepakdd2/lenet-5-imp

lementation-on-mnist-in-pytorch-c6f2ee306e37

[7] Matplotlib, *matplotlib.pyplot.plot*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

[8] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning, Chapter 14: Autoencoders*. MIT Press. https://www.deeplearningbook.org/contents/autoencoders.html

[9] Bengio, Y., Leonard, N., and Courville, A. (2013). *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. arXiv:1308.3432. https://arxiv.org/abs/1308.3432

[10] PyTorch, *Conv2d*. https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

[11] PyTorch, *ConvTranspose2d*. https://docs.pytorch.org/docs/stable/generated/torch.nn.ConvTranspose2d.html

[12] PyTorch, *MSELoss*. https://docs.pytorch.org/docs/stable/generated/torch.nn.MSELoss.html

[13] scikit-image, *Peak Signal-to-Noise Ratio*. https://scikit-image.org/docs/stable/api/skimage.metrics.html

[14] PyTorch, *torch.nn.functional*. https://docs.pytorch.org/docs/stable/nn.functional.html

[15] PyTorch, *Autograd Mechanics*. https://docs.pytorch.org/docs/stable/notes/autograd.html

[16] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. University of Toronto. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[17] PyTorch, *CIFAR10 Dataset*. https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.CIFAR10.html

[18] Loshchilov, I., and Hutter, F. (2019). *Decoupled Weight Decay Regularization*. International Conference on Learning Representations. https://arxiv.org/abs/1711.05101

[19] PyTorch, *CosineAnnealingLR*. https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html

[20] PyTorch, *GroupNorm*. https://docs.pytorch.org/docs/stable/generated/torch.nn.GroupNorm.html

[21] PyTorch, *SiLU*. https://docs.pytorch.org/docs/stable/generated/torch.nn.SiLU.html